

The Ultimate Web Development Handbook



Flexbox in CSS

Flexbox (Flexible Box Layout) is a powerful layout system in CSS that allows you to **align, space, and distribute elements** easily — especially when building responsive layouts.

It's ideal for **one-dimensional layouts** (either a row or a column).

Getting Started

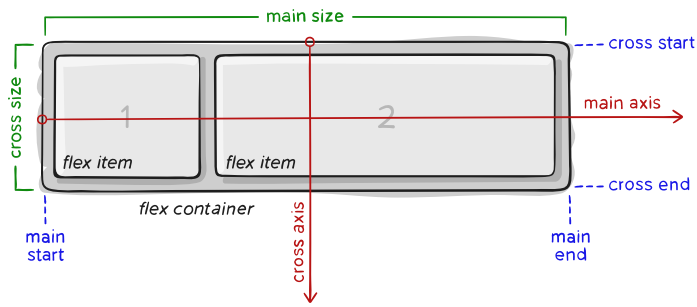
To use Flexbox, set the parent container's `display` to `flex`:

```
.container {  
  display: flex;  
}
```

Now, all **direct children** of `.container` become **flex items**.

Main Concepts

Term	Description
Main Axis	The primary direction (<code>row</code> by default)
Cross Axis	Perpendicular to main axis
Flex Container	The parent element with <code>display: flex</code>
Flex Items	The children inside the container



Flex Direction

Controls the direction of flex items.

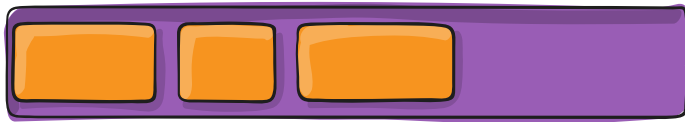
```
.container {  
  display: flex;  
  flex-direction: row;      /* default */  
  flex-direction: row-reverse;  
  flex-direction: column;  
  flex-direction: column-reverse;  
}
```

Justify Content (Main Axis Alignment)

Controls how items are aligned along the main axis (horizontal by default).

```
.container {  
  justify-content: flex-start; /* default */  
  justify-content: flex-end;  
  justify-content: center;  
  justify-content: space-between;  
  justify-content: space-around;  
  justify-content: space-evenly;  
}
```

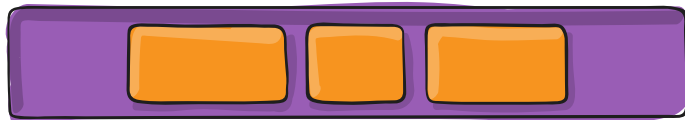
flex-start



flex-end



center



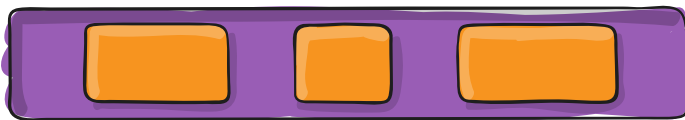
space-between



space-around



space-evenly



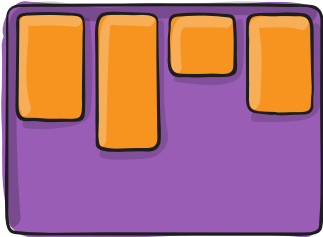
Align Items (Cross Axis Alignment)

Controls how items are **aligned on the cross axis** (vertical by default).

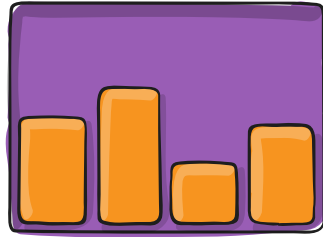
```
.container {  
  align-items: stretch;    /* default */  
  align-items: flex-start;  
  align-items: flex-end;  
  align-items: center;  
}
```

```
align-items: baseline;  
}
```

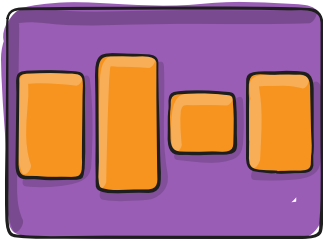
flex-start



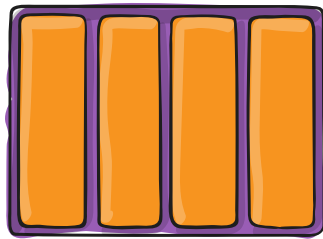
flex-end



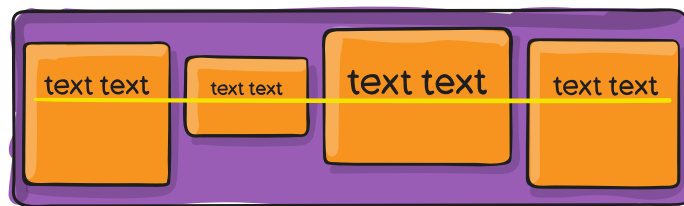
center



stretch



baseline



Align Self

Allows individual items to override `align-items`.

```
.item {  
  align-self: flex-end;  
}
```

Flex Wrap

By default, items try to fit into a single line. Use `flex-wrap` to wrap them:

```
.container {  
  flex-wrap: wrap;  
  flex-wrap: nowrap;      /* default */  
  flex-wrap: wrap-reverse;  
}
```

Gap (Spacing Between Items)

```
.container {  
  gap: 20px;  
}
```

This replaces the need for margins between flex items.

Flex Grow, Shrink, Basis

Control how items grow, shrink, or have an initial size:

```
.item {  
  flex-grow: 1;    /* takes remaining space */  
  flex-shrink: 1;  /* shrink if needed */  
  flex-basis: 200px; /* default size */  
}
```

Shorthand:

```
.item {  
  flex: 1 1 200px;  
}
```

Example Layout

```
<div class="container">  
  <div class="item">One</div>  
  <div class="item">Two</div>  
  <div class="item">Three</div>  
</div>
```

```
.container {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
  gap: 10px;  
}  
.item {  
  background: lightgray;  
  padding: 20px;  
  flex: 1;  
}
```

Summary

- `display: flex` turns a container into a Flexbox layout.
- Use `justify-content`, `align-items`, and `flex-direction` to control layout flow.
- `flex` shorthand (`grow shrink basis`) gives you fine-grained sizing control.
- Use `gap` instead of margins for consistent spacing.

CSS Grid

CSS Grid Layout is a **two-dimensional** layout system that allows you to design web pages in **rows and columns**. It gives you complete control over both axes, unlike Flexbox which is mostly one-dimensional.

Enabling Grid

Set the container's `display` property to `grid` :

```
.container {  
  display: grid;  
}
```

All direct children of this container become **grid items**.

Defining Rows and Columns

You use `grid-template-columns` and `grid-template-rows` to define the grid structure:

```
.container {  
  display: grid;  
  grid-template-columns: 200px 1fr 1fr;  
  grid-template-rows: 100px auto;  
}
```

- `1fr` means "1 fraction of remaining space"
- You can mix fixed units (e.g. `px`) with flexible ones (`fr`)

Repeat Syntax

To avoid repeating values:

```
.container {  
  grid-template-columns: repeat(3, 1fr);  
}
```

Creates 3 equal-width columns.

Grid Gap

Adds spacing between rows and columns:

```
.container {  
  gap: 20px; /* shorthand for row-gap and column-gap */  
}
```

Placing Items

You can control where an item appears in the grid using `grid-column` and `grid-row`.

```
.item {  
  grid-column: 1 / 3; /* spans column 1 to 2 (exclusive of 3) */  
  grid-row: 2 / 3;  
}
```

You can also use `span` :

```
.item {  
  grid-column: span 2;  
}
```

Named Areas (Optional but Powerful)

Define areas using `grid-template-areas` :

```
.container {  
  display: grid;  
  grid-template-areas:  
    "header header"  
    "sidebar content"  
    "footer footer";  
  grid-template-columns: 1fr 3fr;  
  grid-template-rows: auto 1fr auto;  
}
```

Then assign each item:

```
.header { grid-area: header; }  
.sidebar { grid-area: sidebar; }  
.content { grid-area: content; }  
.footer { grid-area: footer; }
```

Auto-Placement

Grid can automatically place items:

```
.container {  
  display: grid;
```

```
grid-template-columns: repeat(auto-fill, minmax(150px, 1fr));  
}
```

This makes the layout responsive, automatically filling space with flexible-width items.

Complete Example

```
<div class="container">  
  <div class="item header">Header</div>  
  <div class="item sidebar">Sidebar</div>  
  <div class="item content">Content</div>  
  <div class="item footer">Footer</div>  
</div>
```

```
.container {  
  display: grid;  
  grid-template-areas:  
    "header header"  
    "sidebar content"  
    "footer footer";  
  grid-template-columns: 1fr 3fr;  
  grid-template-rows: auto 1fr auto;  
  gap: 10px;  
}  
  
.header { grid-area: header; background: #ddd; }  
.sidebar { grid-area: sidebar; background: #bbb; }  
.content { grid-area: content; background: #eee; }  
.footer { grid-area: footer; background: #ccc; }  
  
.item {  
  padding: 20px;  
}
```

Summary

- CSS Grid is **perfect for page layouts** with rows and columns.
- `grid-template-columns` and `grid-template-rows` define structure.
- Use `grid-column` and `grid-row` to place or span items.
- Named grid areas make your layout more readable and semantic.
- Auto-fill and auto-fit allow responsive grids.

CSS Media Queries

Media Queries allow you to create responsive designs by applying CSS rules based on the device's characteristics — such as screen width, height, orientation, and resolution.

They are essential for building **mobile-first, responsive** websites that adapt to various screen sizes (phones, tablets, desktops).

Basic Syntax

```
@media (condition) {  
    /* CSS rules */  
}
```

Example: Target screens smaller than 768px

```
@media (max-width: 768px) {  
    body {  
        background-color: lightgray;  
    }  
}
```

This CSS will apply **only** when the screen width is **768px or less**.

Common Conditions

Media Feature	Description	Example
max-width	Target screens up to a width	<code>@media (max-width: 600px)</code>
min-width	Target screens starting from a width	<code>@media (min-width: 1024px)</code>
orientation	Target <code>portrait</code> or <code>landscape</code> mode	<code>@media (orientation: landscape)</code>
max-height	Target screen height	<code>@media (max-height: 500px)</code>
resolution	Target pixel density	<code>@media (min-resolution: 2dppx)</code>

Responsive Layout Example

```
.container {  
  padding: 20px;  
  font-size: 18px;  
}  
  
@media (max-width: 768px) {  
  .container {  
    padding: 10px;  
    font-size: 16px;  
  }  
}  
  
@media (max-width: 480px) {  
  .container {  
    font-size: 14px;  
  }  
}
```

```
}  
}
```

This approach ensures your layout adjusts smoothly as screen sizes change.

Combining Multiple Conditions

```
@media (min-width: 600px) and (max-width: 1024px) {  
  .sidebar {  
    display: none;  
  }  
}
```

You can combine conditions using `and`, `or`, or `not`.

Mobile-First Approach

Start with styles for small screens, then use `min-width` to add enhancements for larger screens.

```
/* Mobile-first (default) */  
.card {  
  font-size: 14px;  
}  
  
/* Tablet and up */  
@media (min-width: 768px) {  
  .card {  
    font-size: 16px;  
  }  
}  
  
/* Desktop and up */  
@media (min-width: 1024px) {
```

```
.card {  
  font-size: 18px;  
}  
}
```

Media Queries for Print

```
@media print {  
  body {  
    background: white;  
    color: black;  
  }  
  
  .no-print {  
    display: none;  
  }  
}
```

Used to style web pages when printed.

Summary

- Media queries help build **responsive** and **accessible** websites.
- Use `min-width` for mobile-first, scalable layouts.
- Combine media queries for precise control across devices.

CSS Variables and Custom Properties

CSS Variables, also called **Custom Properties**, allow you to store values in a reusable way — making your CSS more maintainable and dynamic.

They follow the pattern of:

```
--custom-name: value;
```

Declaring a CSS Variable

Variables are declared inside a selector using the `--` prefix:

```
:root {  
  --primary-color: #3498db;  
  --font-size: 16px;  
}
```

- `:root` is the highest-level selector (like `html`) — variables here are global.
- Variables declared inside `:root` can be used throughout your stylesheet.

Using a CSS Variable

Use the `var()` function to apply the variable:

```
body {  
  color: var(--primary-color);
```

```
font-size: var(--font-size);  
}
```

Why Use CSS Variables?

☒ Consistency ☒ Easy to update (change in one place) ☒ Theme support (light/dark mode) ☒ Cleaner, scalable CSS

Example: Theming with CSS Variables

```
:root {  
  --bg-color: white;  
  --text-color: black;  
}  
  
body {  
  background-color: var(--bg-color);  
  color: var(--text-color);  
}
```

You can override these in a different class for themes:

```
.dark-theme {  
  --bg-color: #121212;  
  --text-color: #ffffff;  
}
```

Now just add `class="dark-theme"` to `<body>` or a wrapper div to switch themes.

Fallback Values

If a variable isn't defined, you can specify a fallback:

```
h1 {  
  color: var(--heading-color, blue);  
}
```

If `--heading-color` is not set, `blue` will be used instead.

Scoped Variables

Variables can also be scoped to a class or element:

```
.card {  
  --border-radius: 10px;  
  border-radius: var(--border-radius);  
}
```

Only elements within `.card` can access this variable.

Real-World Example

```
:root {  
  --btn-padding: 12px 24px;  
  --btn-color: #fff;  
  --btn-bg: #2ecc71;  
}  
  
.button {  
  padding: var(--btn-padding);  
  color: var(--btn-color);  
  background-color: var(--btn-bg);  
}
```

```
border: none;  
border-radius: 6px;  
cursor: pointer;  
}
```

Update theme by just changing variables in `:root`.

Summary

- Use `--variable-name` to declare and `var(--variable-name)` to use.
- Declare in `:root` for global usage.
- Support theming, dynamic styles, and cleaner code.
- Can be scoped or overridden for flexibility.

CSS Transitions and Animations

CSS provides powerful tools for creating smooth, engaging user experiences through **transitions** and **animations**. These effects can enhance the visual appeal and usability of your website without needing JavaScript.

1. CSS Transitions

A **transition** is used to change CSS properties **smoothly** over a given duration.

Basic Syntax

```
selector {  
  transition: property duration timing-function delay;  
}
```

- **property** : The CSS property to animate (e.g., `background-color` , `transform` , etc.)
- **duration** : How long the transition lasts (e.g., `0.3s` , `1s`)
- **timing-function** : The pace of the transition (`ease` , `linear` , `ease-in` , `ease-out` , etc.)
- **delay** : Optional delay before starting

Example

```
.button {  
  background-color: blue;  
  color: white;  
  transition: background-color 0.3s ease;  
}
```

```
.button:hover {  
  background-color: green;  
}
```

This smoothly changes the button's background color on hover.

Shorthand vs Longhand

Shorthand:

```
transition: all 0.5s ease;
```

Longhand:

```
transition-property: background-color;  
transition-duration: 0.5s;  
transition-timing-function: ease;  
transition-delay: 0s;
```

2. CSS Animations

CSS animations allow more **complex, keyframe-based** changes over time.

Basic Syntax

```
selector {  
  animation: animation-name duration timing-function delay iteration-count  
  direction;  
}
```

Keyframes

Define how the animation should behave at different points:

```
@keyframes slideIn {  
  from {  
    transform: translateX(-100%);  
    opacity: 0;  
  }  
  to {  
    transform: translateX(0);  
    opacity: 1;  
  }  
}
```

Example

```
.box {  
  width: 100px;  
  height: 100px;  
  background-color: red;  
  animation: slideIn 1s ease-in-out;  
}
```

Animation Properties

Property	Description
animation-name	Name of the @keyframes to use
animation-duration	How long the animation takes
animation-delay	Delay before starting
animation-iteration-count	Number of times to run (or infinite)
animation-direction	normal , reverse , alternate
animation-fill-mode	Defines final state: forwards , backwards , both

Property	Description
<code>animation-play-state</code>	<code>running</code> or <code>paused</code>

Looping Animations

```
.pulse {  
  animation: pulse 2s infinite;  
}  
  
@keyframes pulse {  
  0%, 100% {  
    transform: scale(1);  
  }  
  50% {  
    transform: scale(1.1);  
  }  
}
```

Combining Transitions and Animations

Transitions are great for hover and interactive effects. Animations are better for more **dynamic**, self-running effects.

Example using both:

```
.card {  
  transition: transform 0.3s;  
}  
  
.card:hover {  
  transform: scale(1.05);  
}  
  
@keyframes fadeIn {
```

```
from { opacity: 0; }  
to { opacity: 1; }  
}  
  
.card {  
  animation: fadeIn 1s ease;  
}
```

Summary

- Use **transitions** for smooth changes on hover, focus, etc.
- Use **animations** for keyframe-driven effects like entrance, bounce, etc.
- Keep animations subtle and purposeful — avoid overwhelming the user.

CSS Transformations

CSS **transforms** allow you to visually manipulate elements by rotating, scaling, skewing, or translating them. Transforms are applied using the `transform` property.

1. `transform` Property

Syntax:

```
selector {  
  transform: function(value);  
}
```

Multiple functions can be combined:

```
transform: translateX(50px) rotate(45deg) scale(1.2);
```

2. Types of Transformations

a. `translate()`

Moves an element from its current position.

```
.box {  
  transform: translateX(50px); /* Move 50px to the right */  
}
```

Other variations:

- `translateY(30px)` – moves vertically
 - `translate(50px, 30px)` – moves on both axes
-

b. `rotate()`

Rotates the element clockwise by default.

```
.box {  
  transform: rotate(45deg); /* Rotate 45 degrees */  
}
```

Use negative values to rotate counter-clockwise:

```
transform: rotate(-45deg);
```

c. `scale()`

Scales the size of an element.

```
.box {  
  transform: scale(1.5); /* Increase size by 1.5x */  
}
```

- `scaleX(2)` – scales horizontally
 - `scaleY(0.5)` – scales vertically
-

d. `skew()`

Slants an element along the X and/or Y axis.

```
.box {  
  transform: skew(20deg, 10deg); /* Skew in X and Y */  
}
```

Individual axis:

- `skewX(20deg)`
 - `skewY(10deg)`
-

e. `matrix()`

A shorthand to apply multiple transformations using a 2D matrix. Rarely used directly because it's less readable.

3. Transform Origin

By default, transforms are applied relative to the **center** of the element. You can change this with `transform-origin`.

```
.box {  
  transform: rotate(45deg);  
  transform-origin: top left;  
}
```

4. Combining Multiple Transforms

```
.box {  
  transform: translateX(100px) rotate(30deg) scale(1.2);  
}
```

The **order matters**: transforms are applied from left to right.

5. 3D Transforms (Intro Only)

- `rotateX()` , `rotateY()` , and `rotateZ()` add 3D rotation.
- `perspective` property is needed to see 3D depth.

Example:

```
.box {  
  transform: rotateY(45deg);  
  transform-style: preserve-3d;  
}
```

Summary

Transform Function	Description
<code>translate()</code>	Moves element
<code>rotate()</code>	Rotates element
<code>scale()</code>	Resizes element
<code>skew()</code>	Slants element
<code>matrix()</code>	Combines multiple transforms

CSS Transforms are foundational for building modern UI effects — often combined with **transitions** and **animations**.